

Project Contributors — Team Task Assignments & Presentation Prep

Team Overview

This project is a Student Record System in C. The team is divided into 15 roles. Each pair of persons implements the same function; the best version is selected. All persons review the group leader's work to understand the architecture. Two shared reviewers per task pair ensure quality.

Task Assignments (Difficulty: Hardest → Simplest)

Person	Task	Function	Reviewers
1	Group Leader	main (CLI orchestration, menu loop, memory cleanup)	Everyone reviews
2, 3	Load from File	loadFromFile (CSV parsing, array resize, data population)	Persons 1, 4
4, 5	Add Student	addStudent (input validation, duplicate detection, resize)	Persons 3, 6
6, 7	Sort Students	sortStudents , compareMarks (qsort, ascending/descending)	Persons 5, 8
8, 9	Modify Student	modifyStudent (find, partial updates, preserve blanks)	Persons 7, 10
10, 11	Save to File	saveToFile (CSV export, headers, pass/fail status)	Persons 9, 12
12, 13	Initialize System	initializeSystem (allocate array, capacity, error handling)	Persons 11, 14
14, 15	Search Student	searchStudent (locate, display, not-found handling)	Persons 13, 2

Person	Task	Function	Reviewers
2, 3	Display All Students	displayAllStudents (format table, compute PASS/FAIL)	Persons 1, 4
4, 5	Remove Student	removeStudent (locate, shift array, decrement count)	Persons 3, 6
6, 7	Calculate Average	calculateAverageMarks (sum, average, display)	Persons 5, 8
8, 9	Display Menu	displayMenu (print menu options)	Persons 7, 10
10, 11	Welcome Banner	displayWelcomeMessage (program banner)	Persons 9, 12
12, 13	Input Cleanup	clearInputBuffer (consume trailing stdin)	Persons 11, 14

*Alt task (fallback if primary task completed early)

Team Process

1. **Everyone studies Person 1's main function** to understand program flow and architecture.
2. **Paired persons** (2 implementing `loadFromFile`, 3 also implementing `loadFromFile`) write code independently.
3. **Shared reviewers** (Persons 1 and 4) review BOTH implementations, compare quality, and recommend the best version.
4. **Best version selected** for integration based on: code quality, error handling, test results, clarity.
5. **Final code** is one integrated `student_record.c` with all best implementations.

Final Documentation Structure

1. **Table of Contents** — all sections with page numbers
2. **Group Details** — group number, member names, student ID
3. **Requirements** — original problem statement
4. **Planning and Considerations** — modular approach, data model, code review strategy

5. **Program Flow and User Story** — Program Flow diagram, how the program runs(User Story),
 6. **Program/Code** — full `student_record.c` code
 7. **Quick Code Explanation** — 1-2 paragraphs per function: what it does, design choices, edge case handled
 8. **Conclusion**
-

Preparation Instructions

For All Team Members:

1. **Read** `c_tutorial.MD` (if new to C), then `student_record_commented.c` (learn structure).
2. **Study** `student_record.c` carefully.
3. **For your assigned task:** Read both implementations (yours and your pair's), understand the differences.
4. **Prepare ONE slide** covering:
 - Function name and what it does
 - Why the design choices were made (safety, efficiency, user experience)
 - One edge case it handles well
 - A short demo command (e.g., “I will run option 5 to remove a student”)
5. **Test your demo:** Compile with `gcc -o student_record student_record.c`, run your function, time it (10–30s).
6. **Save your slide** as `presentation/person_<N>_task.pdf` or `.pptx`.

For Reviewers:

- Read the two paired implementations carefully.
- Write or run one simple test to verify correctness.
- Check for input validation, memory safety, edge cases.
- Provide feedback to your pair.

For Person 1 (Group Leader):

- Ensure all final implementations are integrated and tested.
 - Coordinate team presentations and demo rehearsal.
 - Present the architecture and integrated demo first (add → load → display → sort → save → exit).
-

How We All Worked Together

The team followed a **modular, pair-based approach**: the group leader established the data model and main flow. Each pair independently implemented the

same function; two shared reviewers compared both versions and selected the best based on quality and correctness. Everyone studied the leader's code to understand integration. All implementations were merged into a single, cohesive `student_record.c`. Each person prepared a presentation slide and demo to explain their function's design choices and edge cases, ensuring every team member contributed meaningfully to both code and presentation.

Contacts

- **Person 1 (Leader):** Architecture, integration, rehearsal coordination.
- **Your 2 Shared Reviewers:** Code quality, testing, demo validation.
- **Everyone:** Prepare slides, study the code, participate in rehearsal.

Goal: Make it clear to the lecturer that every person contributed to design, implementation review, and presentation quality.